TECHNICAL UNIVERSITY OF CHEMNITZ

INVOX

Multi-Agent Template Filling using LLMs

Design and Comparative Evaluation with Speech Input Case Study

PRESENTED BY
Huzefa Ismail Jadliwala
Mat. No. 806286

SUPERVISORS
Dr. Christoph Carl Kling (External)
Verena Traubinger (Internal)

EXAMINER
Dr. Sebastian Heil

DATE
March 18, 2026

Good morning everyone, and thank you for being here. My name is Huzefa Ismail Jadliwala, and today I'll be presenting my master's thesis on using AI agents to automatically fill in structured templates from spoken language.

This work was done in collaboration with AIConver, with external supervision from Dr. Christoph Carl Kling, and academic supervision from Verena Traubinger here at TU Chemnitz.

The system I built is called **Invox**. The core idea is simple — instead of someone manually typing information into a form or template, they just speak, and Invox figures out what goes where. Over the next thirty minutes I'll cover why this is a hard problem, how I designed the system, and what the evaluation results showed.

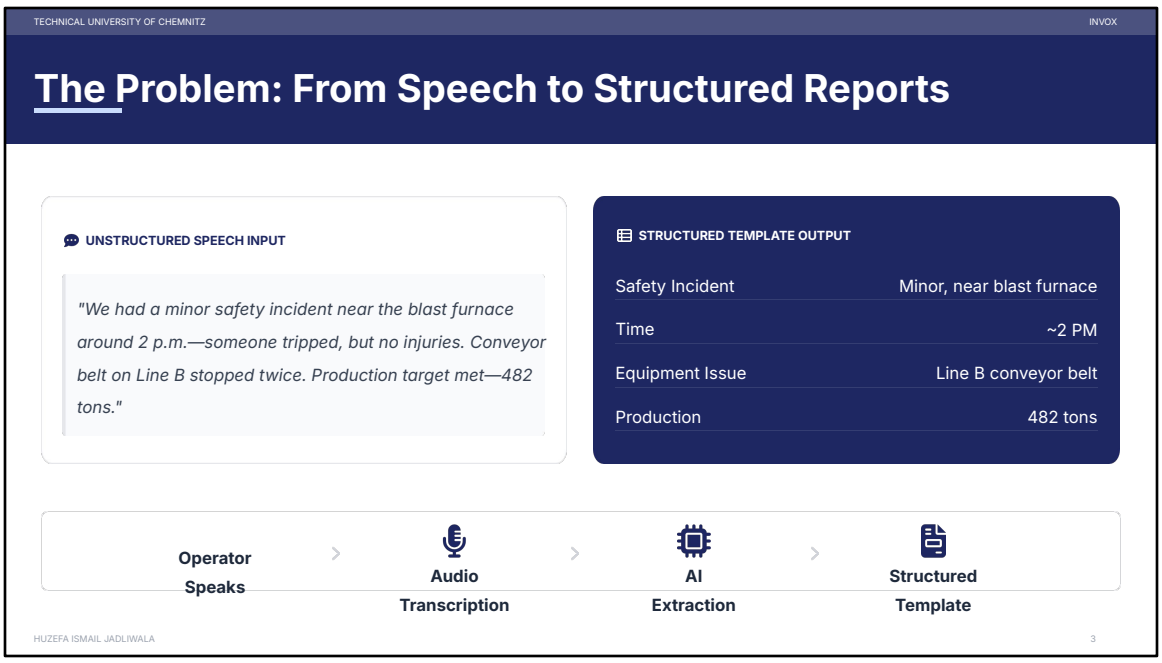
admin@aiconver.com

The Problem: From Speech to Structured Reports

Imagine you are an operator at a steel plant. Your shift just ended. You are tired. You are wearing thick gloves. The machines around you are loud.

Now someone asks you to fill in a template. Equipment issues. Safety incidents. Production numbers. All typed manually.

That is slow. It is error-prone. And in that environment — it is really impractical.



The simple solution? Just let the operator speak. And let the system fill in the template automatically.

That is exactly what Invox does. The operator speaks. The audio gets transcribed. The AI pulls out the relevant information. And the template gets filled — like you see on the right side of this slide.

But here is the thing — converting messy, unstructured speech into precise, structured template fields is actually a very hard problem. And that is what this thesis is about.

So why not just send the transcript to one AI model and ask it to fill the template? That is the obvious approach. And most current systems do exactly that.

But it has three big problems.

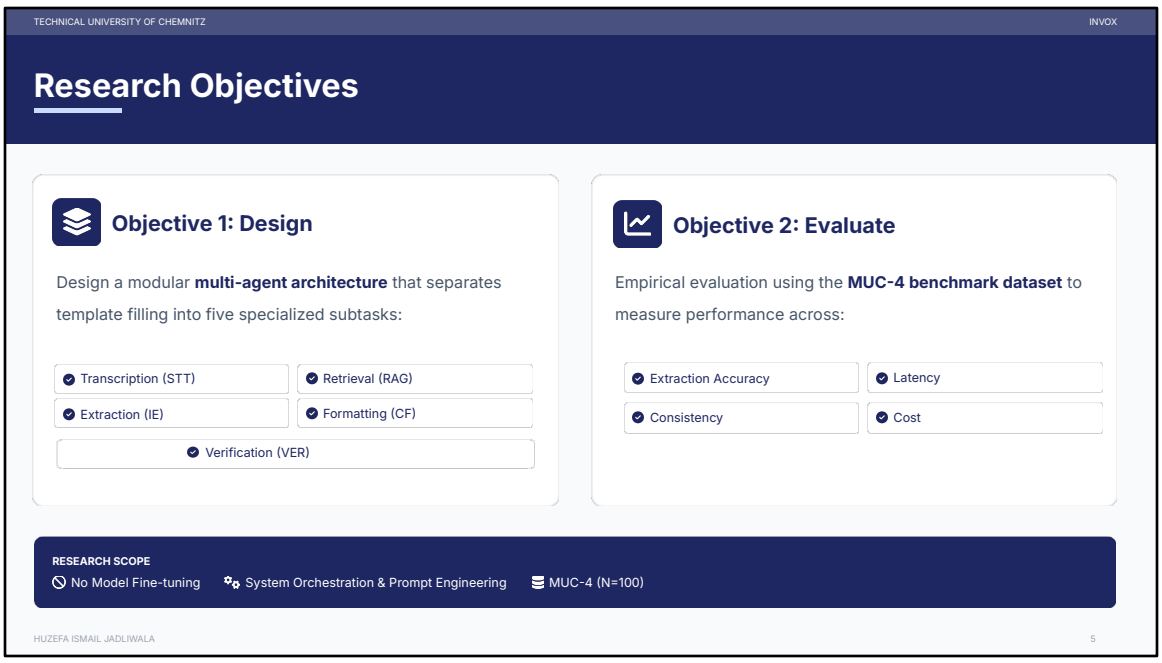
First — these systems are **fragile**. If the input is noisy, or the speaker jumps between topics — which happens a lot in real industrial settings — the whole system can fail.

Second — they are **opaque**. If a field is filled incorrectly, you have no idea why. Was it the transcription? The understanding? The field mapping? You just get a wrong answer with no explanation.

Third — they are **rigid**. Want to support a new domain or a new template structure? You have to retrain the entire model from scratch. That is expensive and slow.

So the gap is clear. No existing system is transparent, correctable, and adaptable at the same time.

That is exactly what Invox is designed to fix.



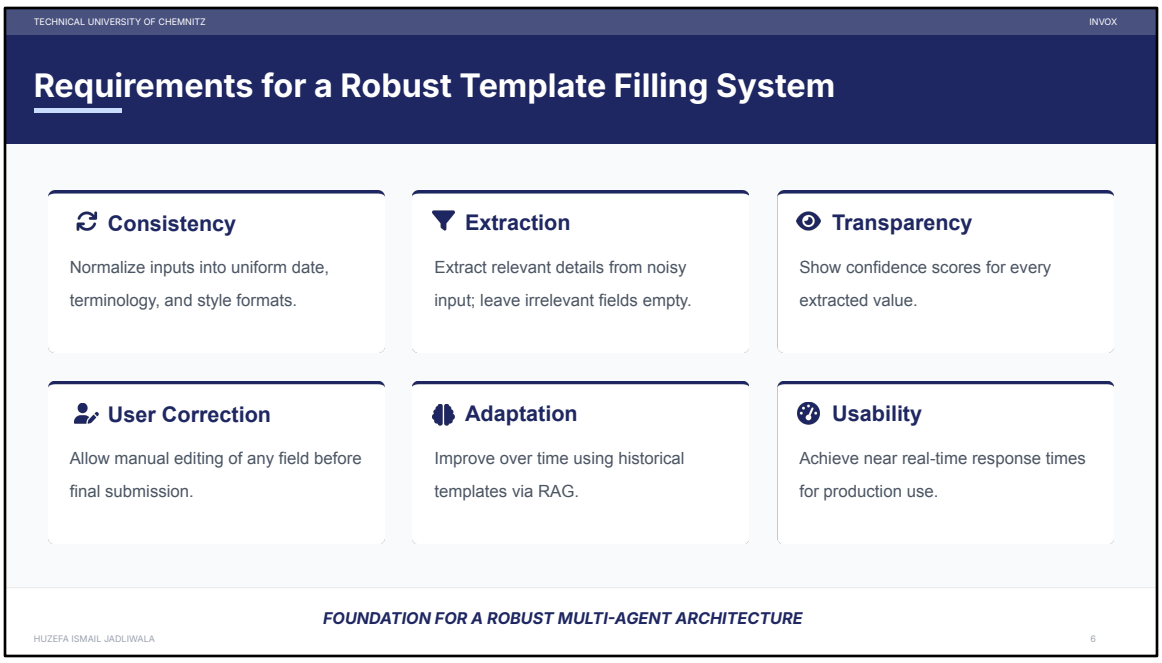
"So what did this thesis actually set out to do? Two clear objectives.

First — **design**. Build a modular multi-agent system that splits template filling into five specialised tasks. Transcription, retrieval, extraction, formatting, and verification. Each handled by a dedicated agent.

Second — **evaluate**. Test it empirically on the MUC-4 benchmark. Measuring three things — extraction accuracy, latency and cost, and robustness to noisy speech input.

To make the comparison meaningful, I built not one version of the system — but four different orchestration strategies. From a simple single-pass approach all the way to a full multi-model consensus mechanism.

And one important scope note — no model fine-tuning was involved. Everything runs on standard GPT-4, Gemini 2.0, and Whisper. All the intelligence comes from prompt engineering and retrieval.



Before designing anything, I needed to establish what a good template filling system actually needs to do. Through literature analysis and real-world input from AIConver, I identified six core requirements.

The first three are about output quality.

R1 — Consistency. Normalise everything into uniform formats. Dates, terminology, naming conventions. So the database stays clean.

R2 — Extraction accuracy. Extract the right information. And just as importantly — leave fields empty when the information is not there.

R3 — Transparency. Every filled field should show where the value came from and how confident the system is.

The next three are about system behaviour over time.

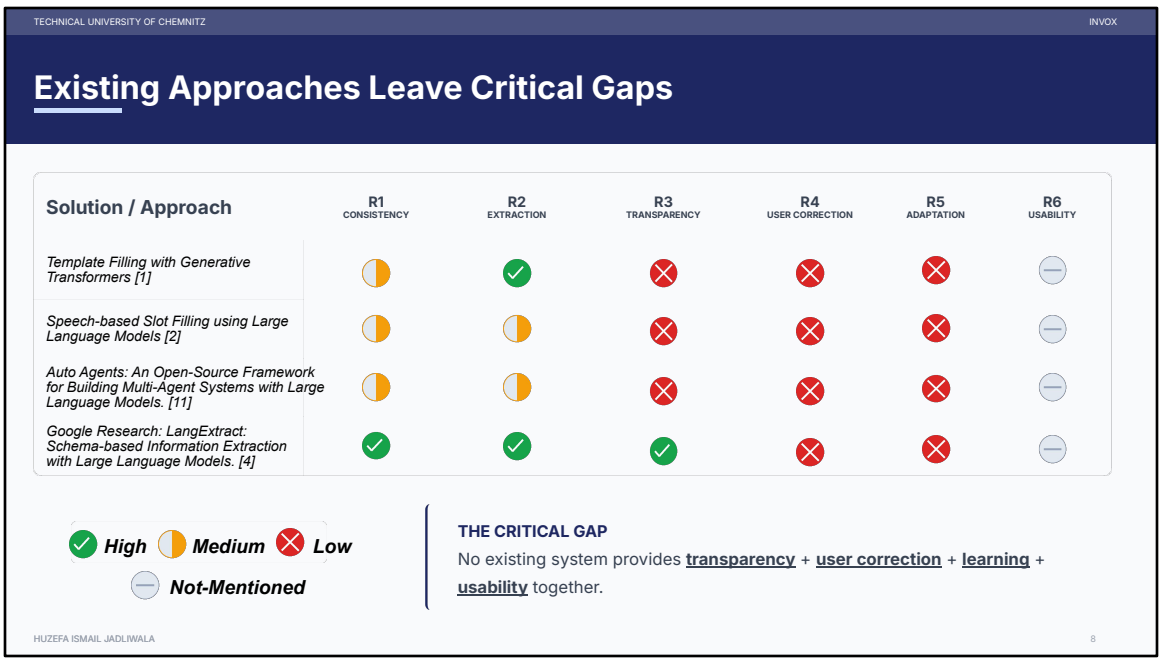
R4 — User correction. Humans can edit any field before submission. The AI assists — but never has the final word.

R5 — Adaptation. The system should get better over time by learning from historical templates through RAG.

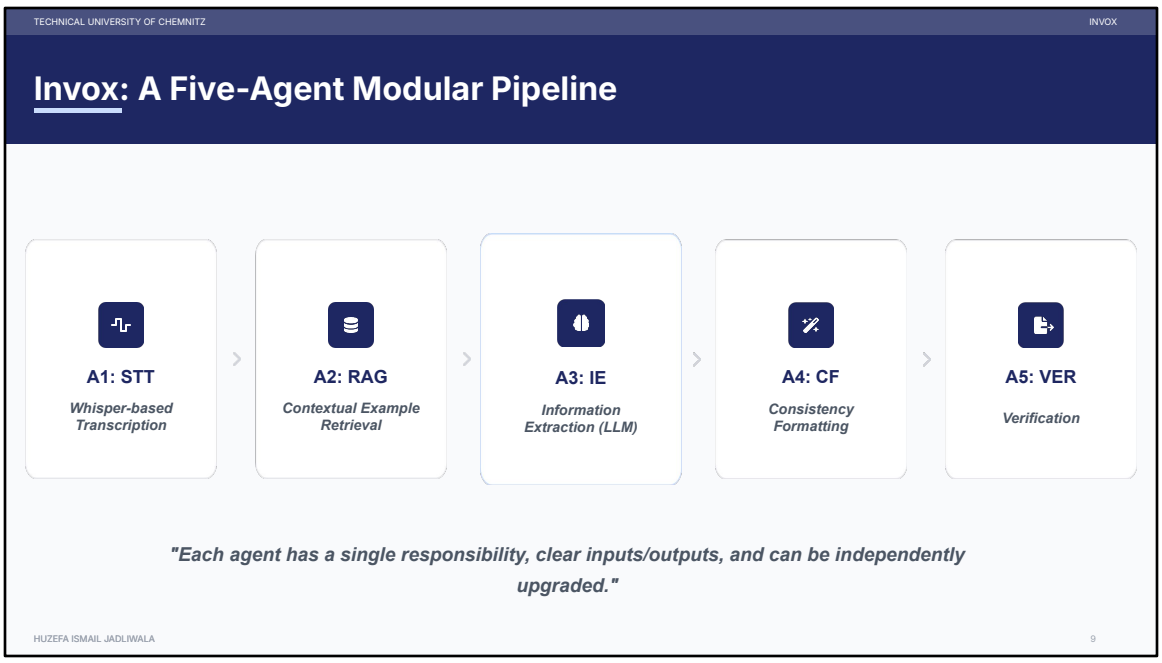
R6 — Usability. Fast enough for real production environments. Near real-time response is a hard requirement.

These six requirements became the foundation for every design decision in this thesis.

Now let's see how existing solutions measure up against our six requirements. Du et al., Sun et al., Wu et al., and Google LangExtract — these are the four most relevant works we found across template filling, prompt engineering, and multi-agent research.



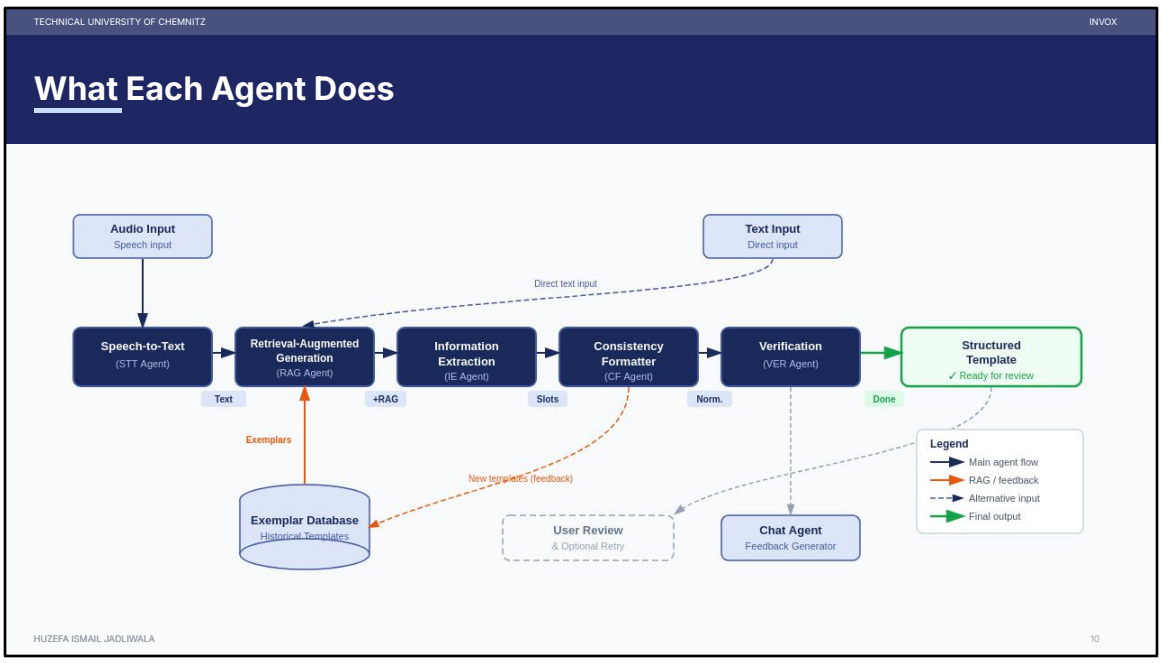
Du et al. does well on extraction.
But no transparency, no user correction, no learning.
Sun et al. and Wu et al. are similar — medium on consistency and extraction, but nothing on the right side.
Google LangExtract is the strongest — high on consistency, extraction, and even transparency.
But still no user correction and no learning. And notice R6 — usability.
Not a single one even mentions it.
The pattern is clear.
The further right you go — the more every solution falls apart. No system covers all six together. That is the critical gap. And that is exactly what motivated Invox.



This is Invox. Instead of one big model doing everything — I split the task into five specialised agents. Each with a single clear responsibility.

Audio or text comes in. It flows through transcription, retrieval, extraction, formatting, and verification. Out comes a structured template — ready for human review.

Each agent has clear inputs and outputs. So if something goes wrong — you know exactly where. And because agents are independent — any one can be swapped without touching the rest.

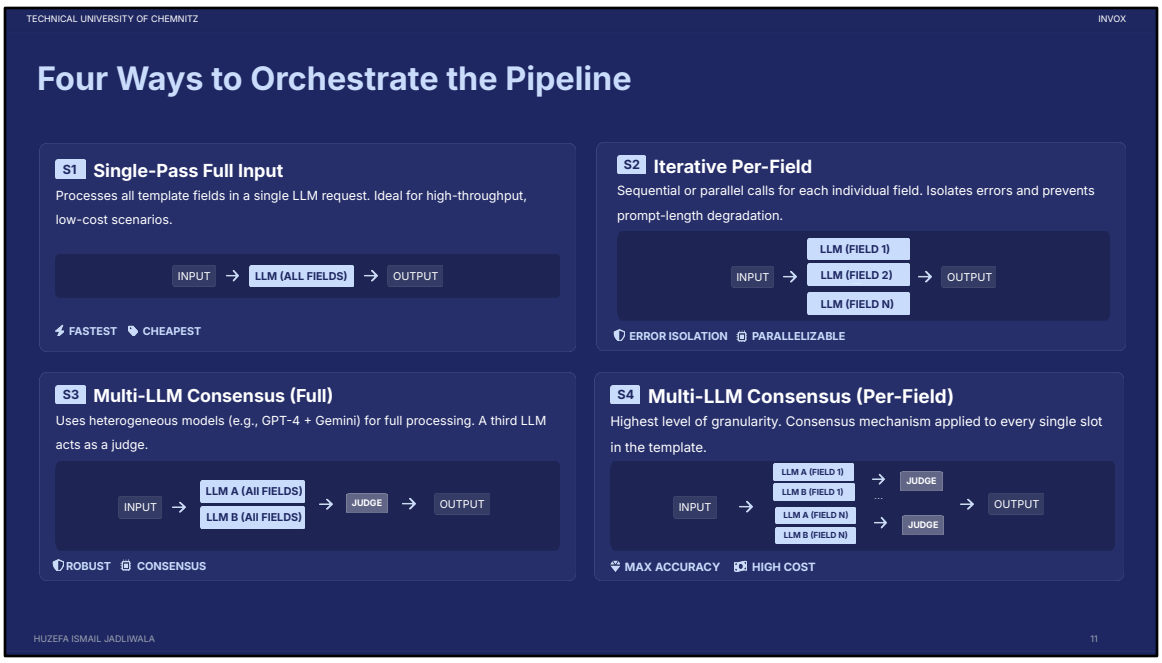


Now let me go one level deeper and walk through exactly what each agent takes in and produces — because this transformation chain is what makes the system both traceable and reliable.

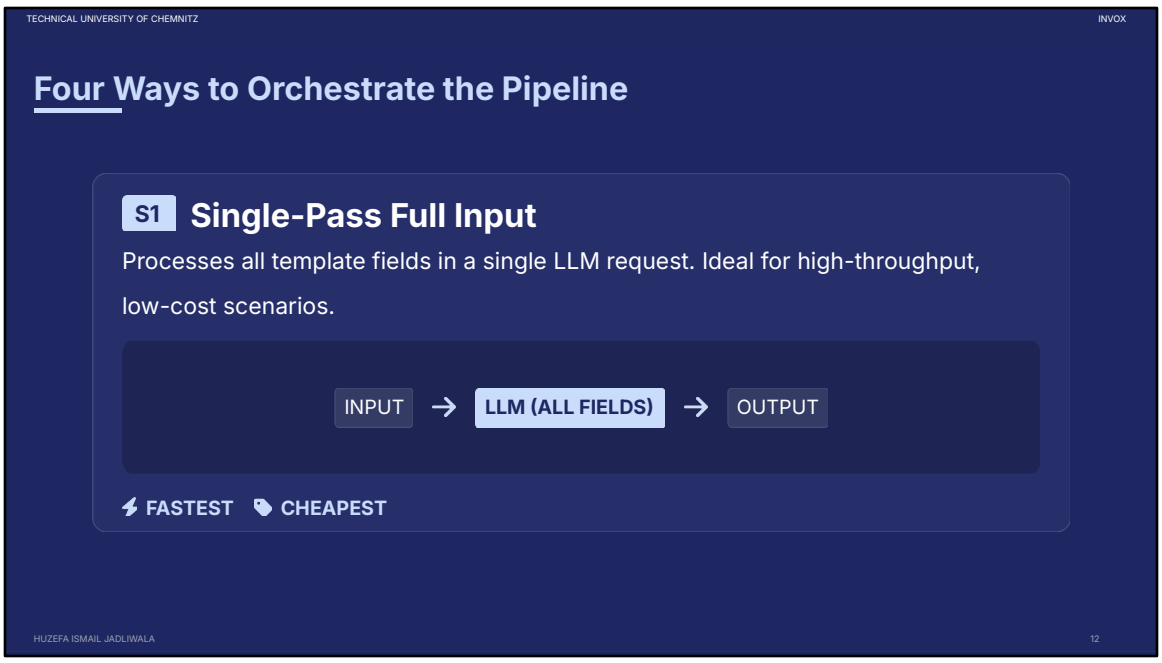
The STT agent takes raw audio and outputs transcribed text with confidence scores and timestamps. The RAG agent takes that text plus the template schema and retrieves the top-k most similar historical templates as few-shot examples. The IE agent — the most critical one — takes all of that and outputs slot-value pairs, crucially outputting null for fields where the information simply isn't present. This is key for controlling hallucinations.

The CF agent then normalizes those raw values — converting informal dates to ISO format, mapping terminology to valid enum values. And the VER agent performs a final quality check, flagging any fields it is uncertain about with a confidence score.

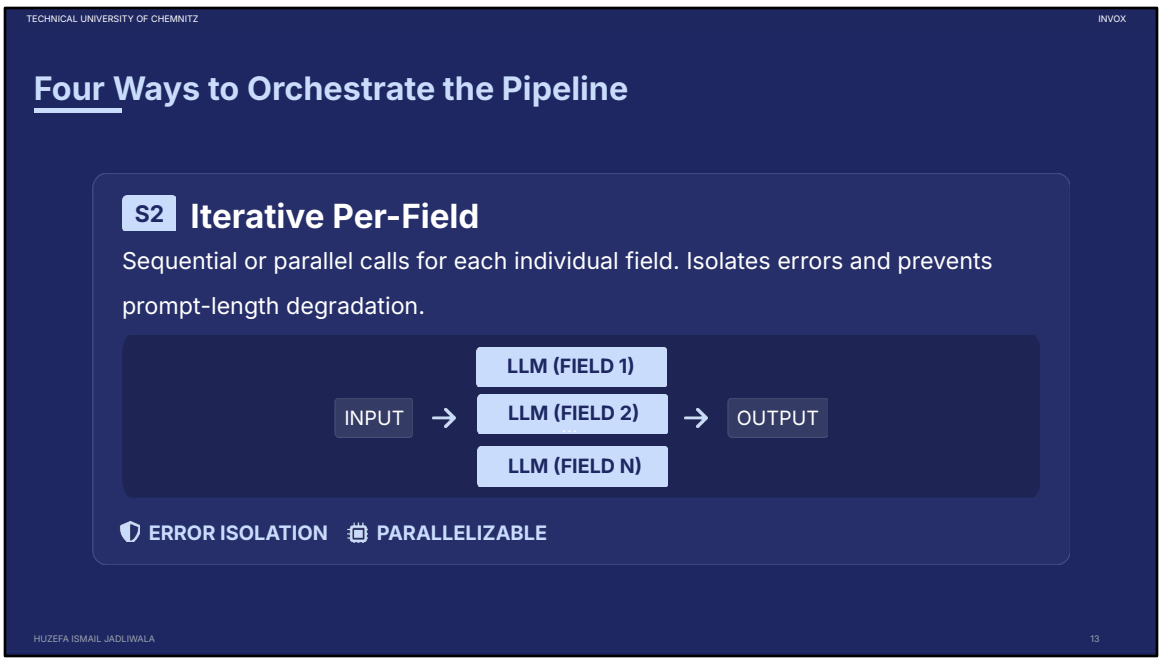
Together these five agents form a clean transformation chain — from noisy speech all the way to a verified, structured, ready-to-review template.



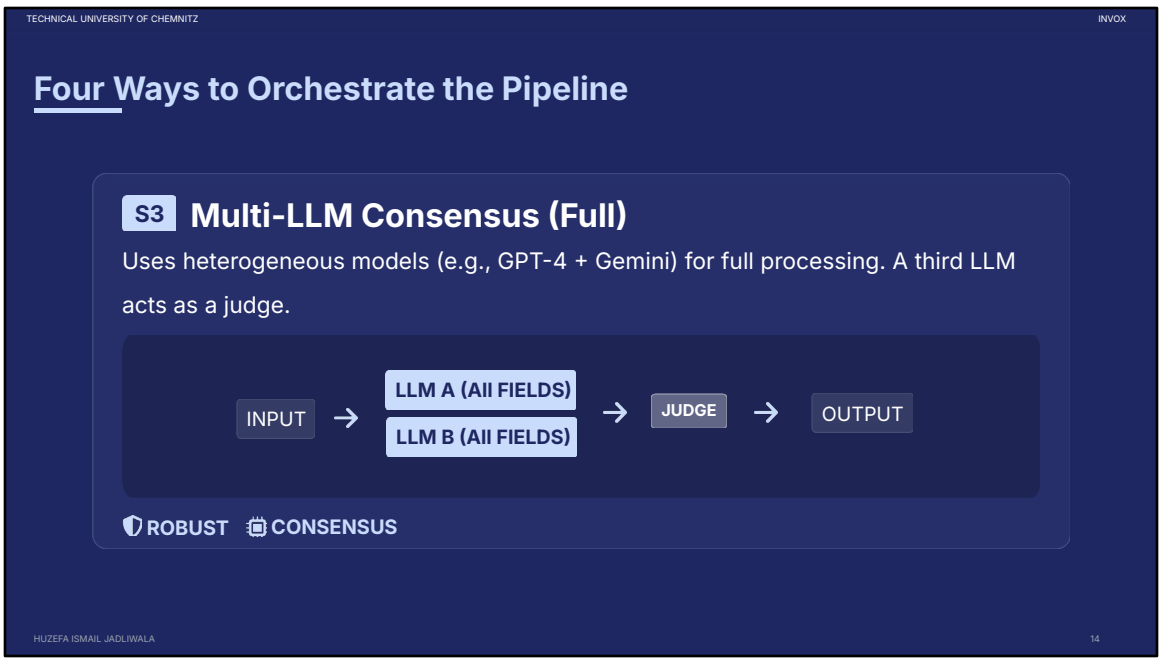
We have the five-agent pipeline — but there is still an important design question. How should the IE agent actually call the LLM?
One field or all fields?
One model or multiple?
I designed four distinct strategies to answer exactly this.



Strategy 1 — Single-Pass Full Input. The LLM receives all template fields at once in a single request and fills them all in one shot. Simplest and fastest

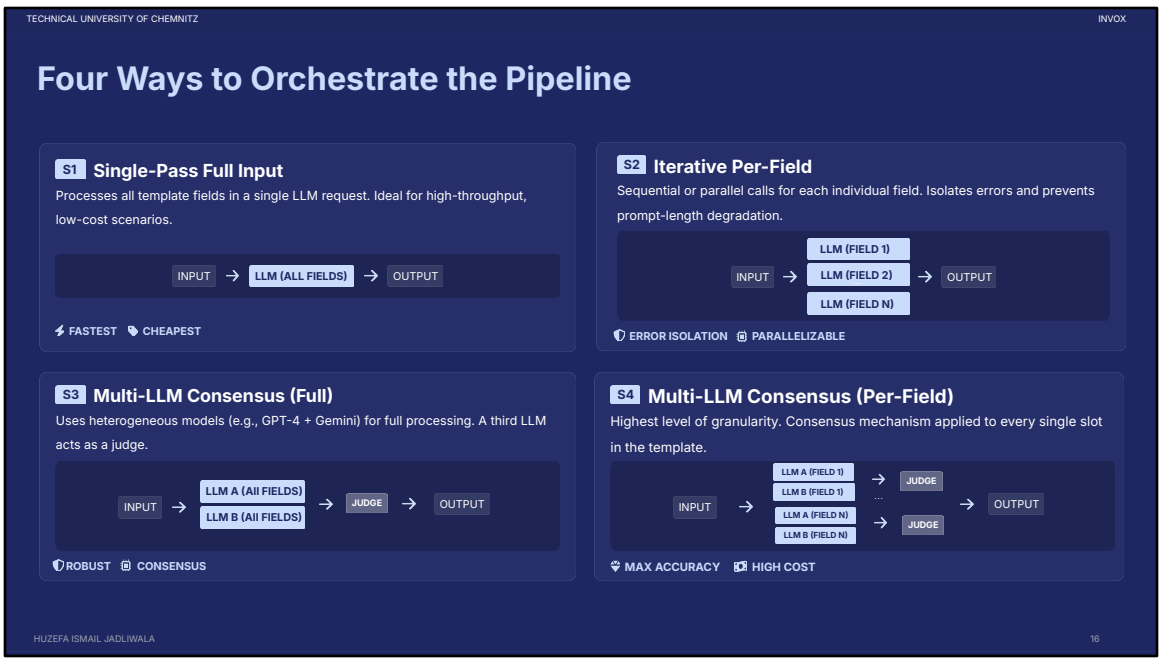


Strategy 2 — Iterative Per-Field. One separate LLM call per field. A failure on one field doesn't affect the others — and the calls can be parallelized.

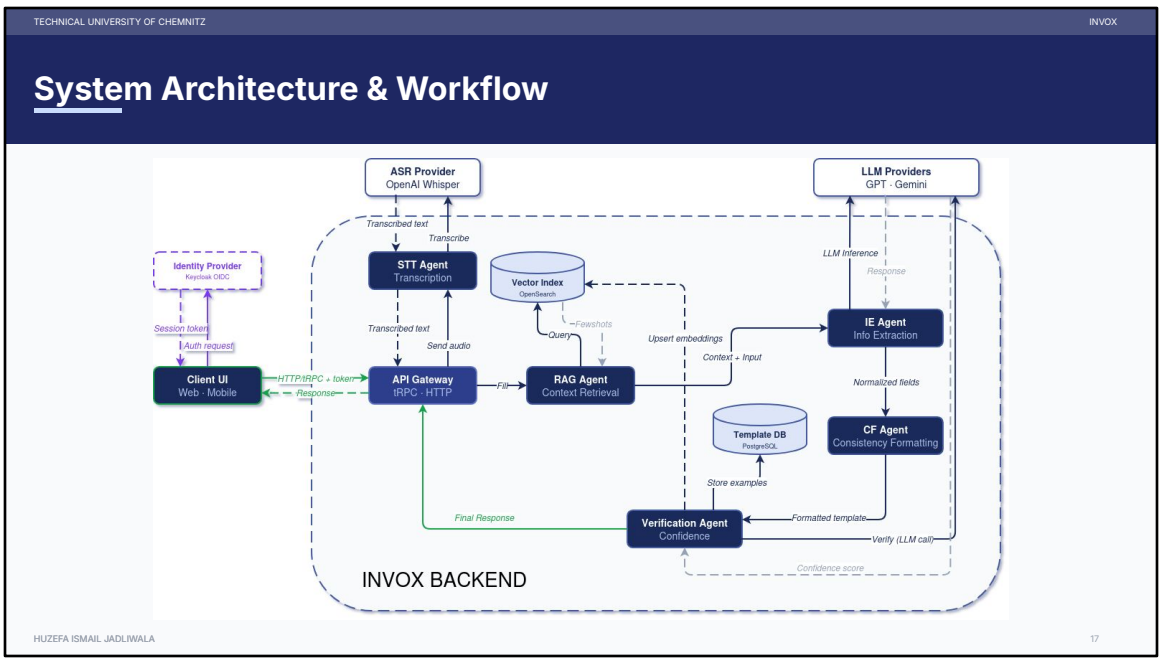


Strategy 3 — Multi-LLM Consensus Full. GPT-4 and Gemini both process the full input independently. A third LLM acts as judge. More robust — but more expensive.

Strategy 4 — same consensus mechanism but at the individual field level. Highest granularity, highest accuracy potential — but also the highest cost and latency.



The key message — same five-agent pipeline in all four cases. Only the orchestration changes. This means the system can be deployed differently depending on the use case and budget.



This is how Invox is actually built. Three layers, stacked on top of each other.

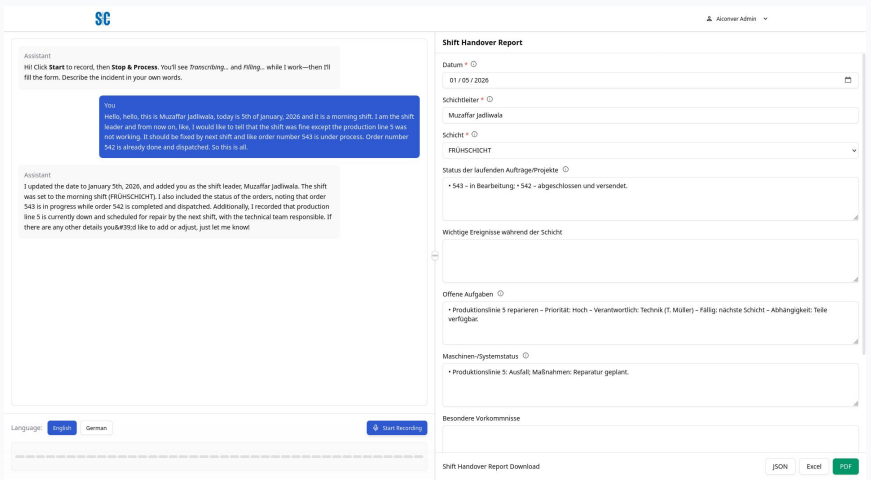
At the top — the React.js web app. That is what the operator sees and interacts with. In the middle — the tRPC API gateway. It connects the frontend to the agents and validates all data at every step. At the bottom — the five agents, each running independently.

They connect to external services — OpenAI and Gemini for language models, Whisper for transcription, OpenSearch for vector retrieval, PostgreSQL for storage, and Keycloak for authentication.

One implementation detail worth highlighting — the RAG setup. Rather than simple keyword search, I used embedding-based semantic search using text-embedding-3-large. So when RAG retrieves historical templates, it finds ones that are semantically similar to the current input — not just ones that share the same words. This is what makes the few-shot examples genuinely useful for the IE agent.

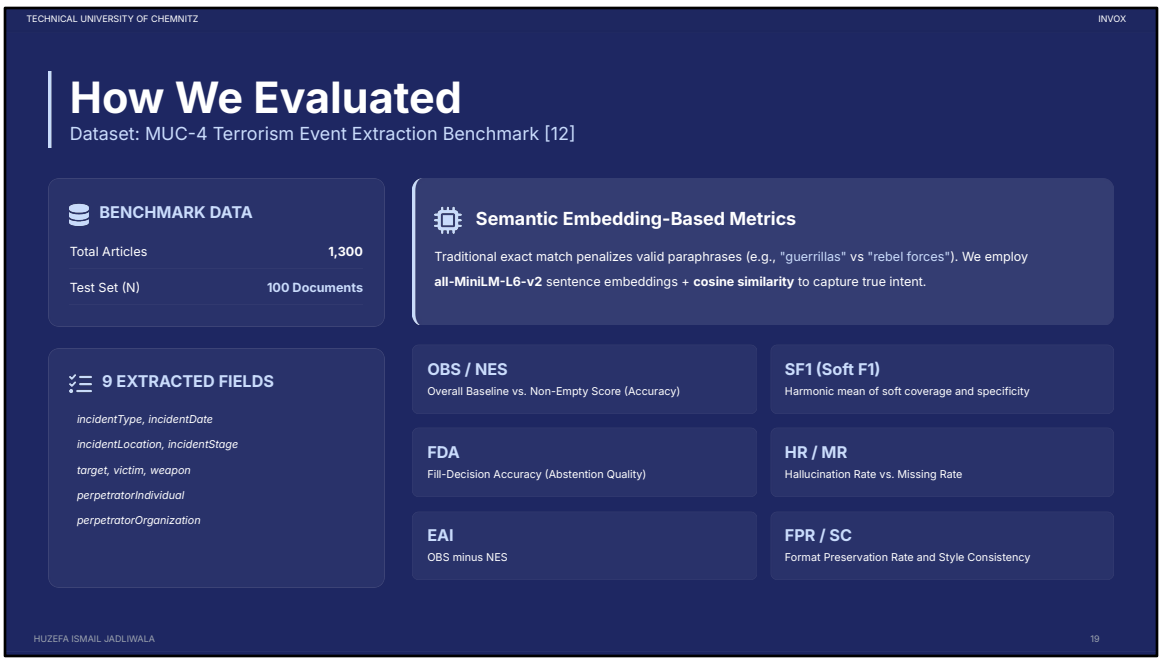
And importantly — no model was fine-tuned. All intelligence comes purely from prompt engineering and retrieval.

Implementation Details



SCAN ME
Web App

This is the actual Invox interface. On the left — the operator speaks, and the system transcribes and processes in real time. On the right — the Shift Handover Report template, automatically filled. You can see the date, shift leader, shift type, order statuses, and machine issues — all extracted from a single spoken input. The operator can review, correct any field, and export as PDF, Excel, or JSON. This is the human-in-the-loop mechanism in action.



Let me walk you through how I evaluated Invox.

The dataset is MUC-4 — a well known benchmark for information extraction. It contains 1,300 news articles about terrorism events. I used 100 documents as my test set.

Each document has nine fields to extract — things like incident type, date, location, victim, and weapon.

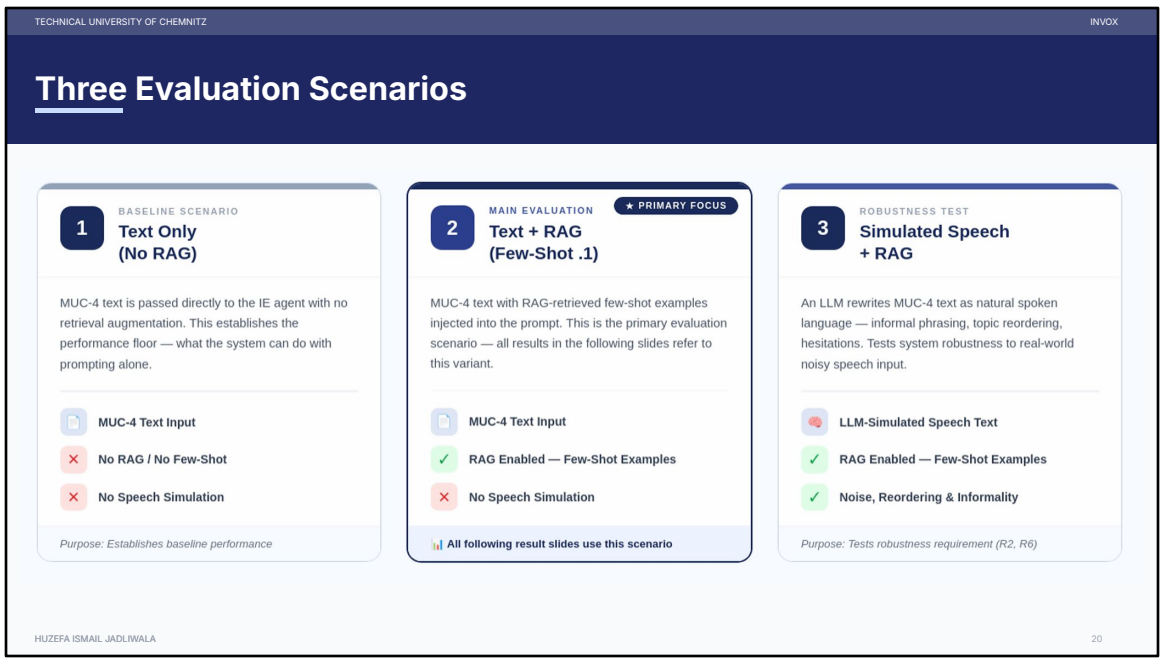
Now the important question — how do you measure if the extraction is correct?

Traditional exact match scoring is too strict. For example — 'guerrillas' versus 'rebel forces' would count as wrong. Even though they mean the same thing. That is unfair — especially for speech input where paraphrasing is natural.

So instead I used **Soft F1** — based on sentence embeddings and cosine similarity. It captures meaning, not just exact wording.

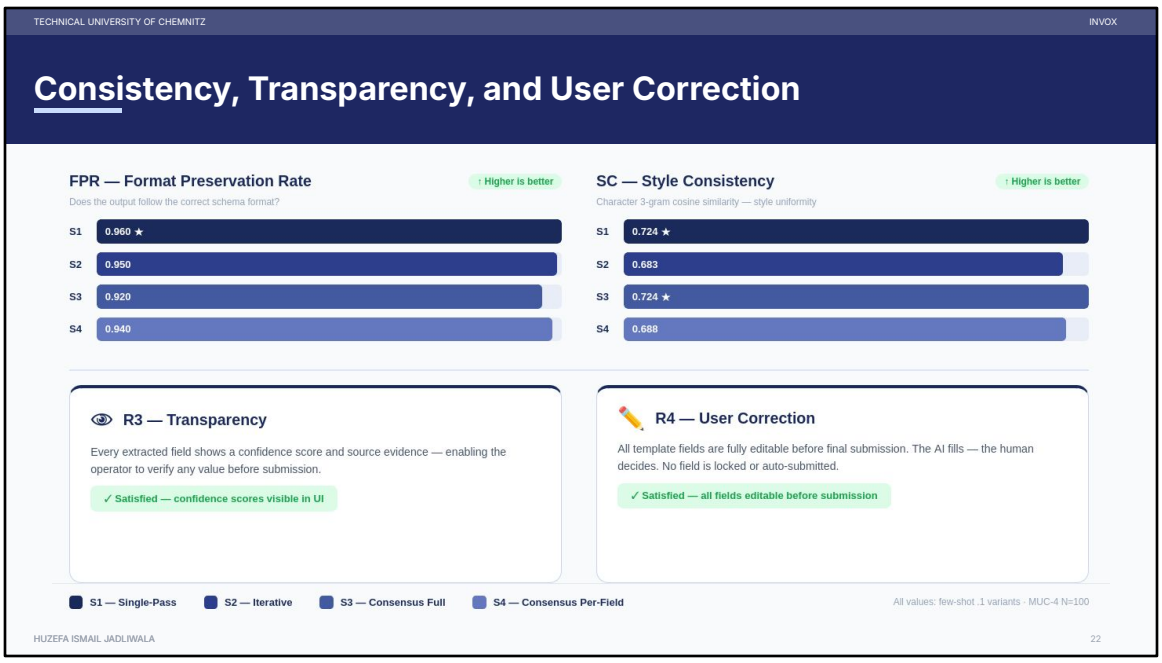
Alongside Soft F1, I tracked five more metrics — overall accuracy, non-empty field accuracy, hallucination rate, missing rate, and fill decision accuracy.

Together these give a much fairer and more complete picture of how the system actually performs.

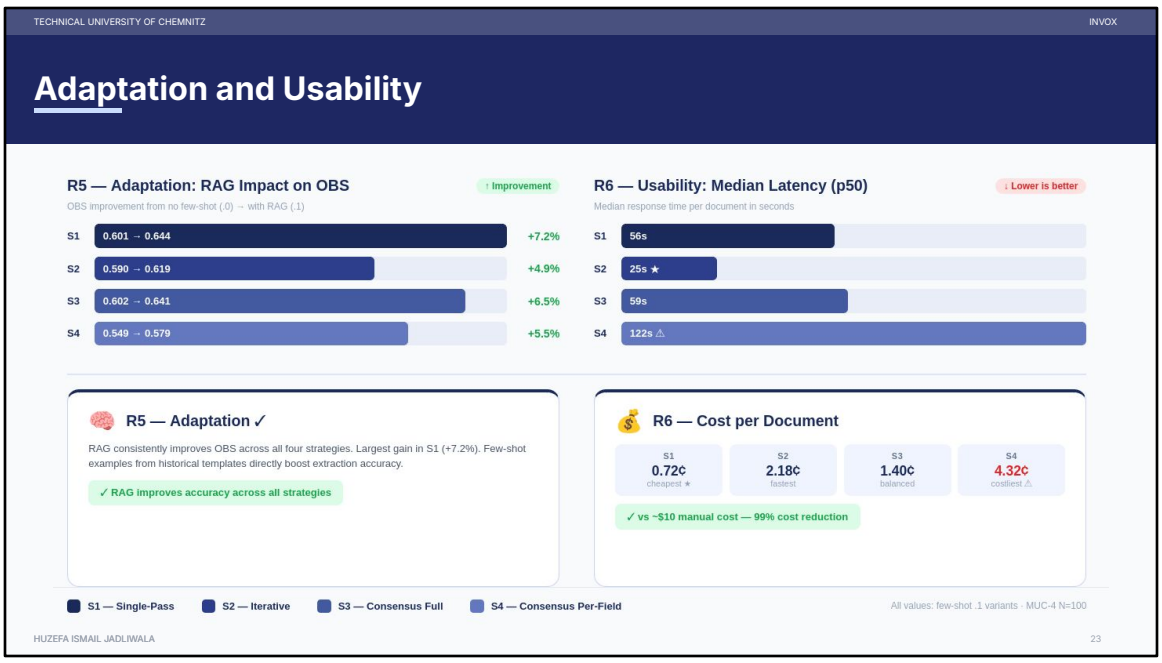


Before I show the results, let me explain how I structured the evaluation.
I tested all four strategies across three different scenarios.
Scenario 1 — text input with no RAG. This is the baseline. No few-shot examples, just the raw prompt.
Scenario 2 — text input with RAG enabled. This is the main focus. The system retrieves similar historical templates and injects them as few-shot examples before extraction.
Scenario 3 — simulated speech with RAG. Here I used an LLM to rewrite the MUC-4 text as natural spoken language — informal phrasing, reordered information, hesitations. This tests how robust the system is to real-world noisy input.
All results you will see in the next slides come from Scenario 2.

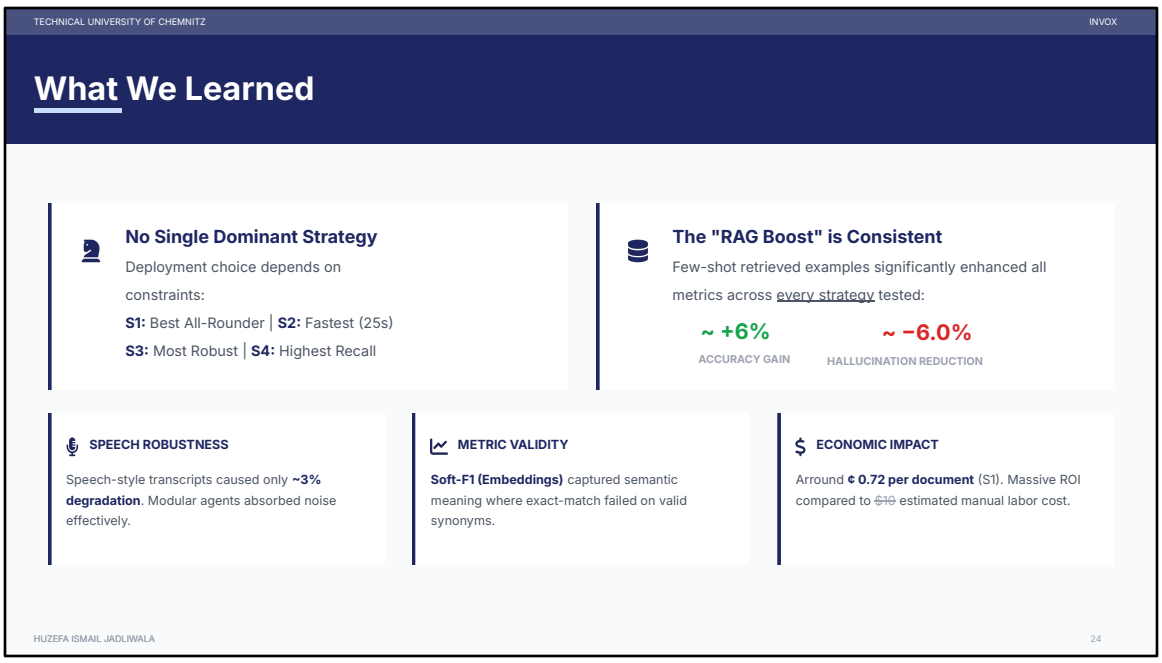
Now let me walk through the extraction accuracy results across all four strategies.
Looking at OBS — overall accuracy — S1 leads with 0.644, closely followed by S3 at 0.641. S4 scores lowest at 0.579.
For NES — accuracy on fields that actually have values — S4 surprisingly leads at 0.624. This means S4 fills the most fields. But this comes at a cost — look at the Hallucination Rate. S4 hits 0.476 — nearly half of its filled fields are wrong. S1 has the lowest hallucination rate at just 0.180.
Soft F1 tells a balanced story — S1 leads at 0.642, but all four strategies are very close.
Fill Decision Accuracy — whether the system correctly decides to fill or leave a field empty — S1 again leads at 0.746.
And Missing Rate — S4 lowest at 0.144, meaning it rarely leaves fields empty. But again — that is because it fills aggressively, which drives the hallucination up.
The key takeaway — S1 is the most reliable all-rounder. S4 maximizes recall but at the cost of precision.



Now let me cover three more requirements.
Starting with R1 — Consistency. The two charts show format preservation rate and style consistency across all four strategies. All strategies exceed 92% format compliance — meaning the output reliably follows the correct schema. S1 and S3 tie for the highest style consistency at 0.724.
R3 — Transparency. Every extracted field in Invox shows a confidence score and the source evidence behind it. The operator can see exactly where each value came from before accepting it. This was demonstrated in the UI demo.
R4 — User Correction. No field is ever auto-submitted. The operator can edit any field before final submission. The AI assists — but the human always has the final word.
So R1, R3, and R4 are all satisfied.



Last two requirements — adaptation and usability.
R5 — Adaptation. The left chart shows the OBS improvement when RAG is enabled versus no RAG. Every single strategy improves. S1 gains the most — plus 7.2%. This directly validates the RAG design decision. Historical templates retrieved as few-shot examples genuinely help the IE agent extract more accurately.
R6 — Usability. Looking at latency — S2 is the fastest at 25 seconds median. S1 and S3 are around 56 to 59 seconds. S4 is the slowest at 122 seconds due to its per-field consensus mechanism.
On cost — S1 is the cheapest at just 0.72 cents per document. S4 is the most expensive at 4.32 cents. But compare this to the manual alternative — a 25 minute operator task at industrial rates costs around 10 dollars per report. Invox delivers a 99% cost reduction.
Both R5 and R6 are fully satisfied.



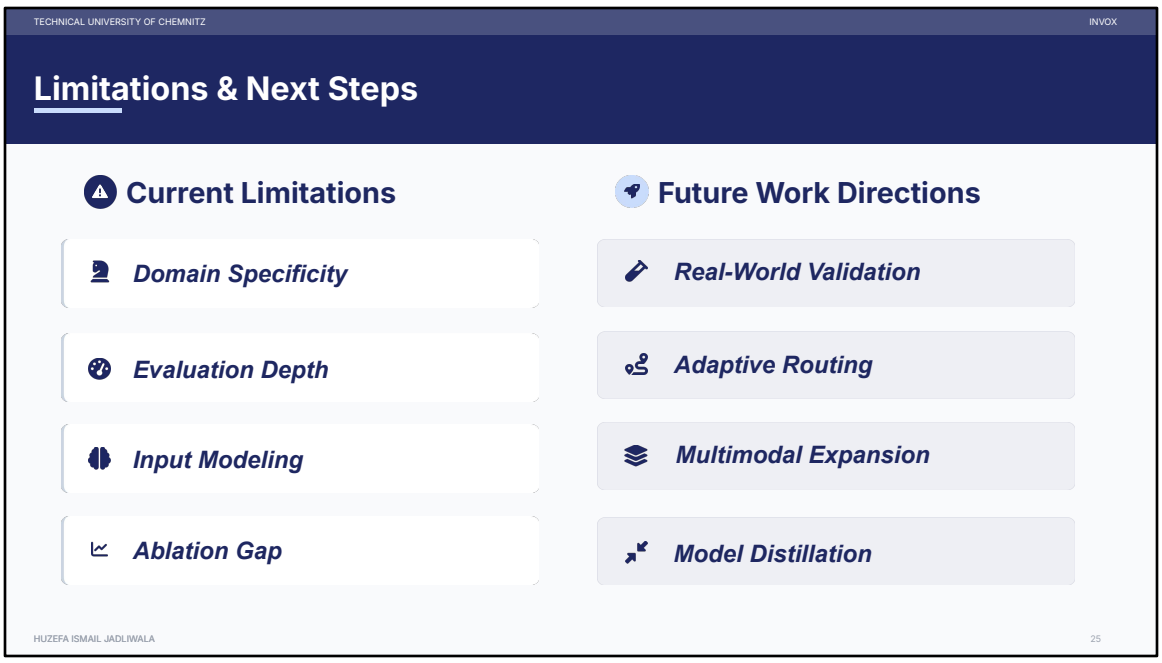
So what did we learn from all of this?

First — no single strategy dominates. S1 is the best all-rounder, S2 is the fastest, S3 the most robust, and S4 maximises recall. The right choice depends on the deployment constraints.

Second — the RAG boost is consistent. Across every strategy tested, few-shot retrieval improved accuracy by an average of 3.5% and reduced hallucination by 6%. This validates the RAG design decision.

Third — speech robustness. When we tested with simulated speech input, performance dropped by only around 3%. The modular agent design absorbed the noise effectively.

And finally — economic impact. At 0.72 cents per document for S1, compared to roughly 10 dollars for manual entry, Invox delivers a 99% cost reduction in production.



Every research has limitations — and I want to be honest about four of them.

First — **domain specificity**. I only tested on MUC-4 — a terrorism dataset. This does not prove Invox works in healthcare or manufacturing. That needs separate validation.

Second — **evaluation depth**. 100 documents is a small test set. There was no qualitative assessment with real operators. A larger real-world study would strengthen the conclusions.

Third — **input modeling**. I simulated speech input by modifying text — not real audio with background noise and accents. So the 3% degradation figure is a best-case estimate. Real factory conditions could be harder.

Fourth — **ablation gap**. I could not isolate the individual contribution of the CF and VER agents. The full pipeline works well — but I cannot yet say exactly how much each of those two agents contribute on their own.

On future work — four directions stand out. Real-world validation with actual operators. ML-driven routing to automatically pick the best strategy based on input complexity. Multimodal expansion to include sensor data and images. And model distillation to compress the pipeline into smaller, cheaper local models.

The good news — because Invox is modular, every single one of these can be added as an independent agent without touching the rest of the system.

References

[1] Du, X., Rush, A.M.: Template filling with generative transformers. ACL Anthology (2021)

[2] Sun, G., Feng, S., Jiang, D., Zhang, C., Gašić, M., Woodland, P.C.: Speech-based slot filling using large language models. arXiv preprint arXiv:2311.07418 (2023)

[3] Le, N.K., Bui, Q.M., Nguyen, M.N., Nguyen, H., Vo, T., Luu, S.T., Nomura, S., Nguyen, M.L.: Automated web application testing: End-to-end test case generation with large language models and screen transition graphs. arXiv preprint arXiv:2506.02529 (2025)

[4] Google Research: LangExtract: Schema-based information extraction with large language models. https://github.com/google-research/langextract (2024)

[5] Brown, T.B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., et al.: Language models are few-shot learners. Advances in Neural Information Processing Systems, vol. 33, pp. 1877–1901 (2020)

[6] Zhang, X., Peng, B., Li, K., Zhou, J., Meng, H.: SGP-TOD: Building task bots effortlessly via schema-guided LLM prompting. arXiv preprint arXiv:2305.09067 (2023)

[7] Peng, B., Galley, M., He, P., Cheng, H., Xie, Y., Hu, Y., Huang, Q., Liden, L., Yu, Z., Chen, W., Gao, J.: Check your facts and try again: Improving large language models with external knowledge and automated feedback. arXiv preprint arXiv:2302.12813 (2023)

[8] Shen, Y., Song, K., Tan, X., Li, D., Lu, W., Zhuang, Y.: HuggingGPT: Solving AI tasks with ChatGPT and its friends in Hugging Face. arXiv preprint arXiv:2303.17580 (2023)

[9] Liang, Y., Qin, Y., Liu, Y., Ye, Y., Liu, Z., Sun, M., et al.: TaskMatrix.AI: Completing tasks by connecting foundation models with millions of APIs. arXiv preprint arXiv:2303.16434 (2023)

[10] Wang, Z., Zhao, Z., Lyu, Y., Chen, Z., de Rijke, M., Ren, Z.: A cooperative multi-agent framework for zero-shot named entity recognition. In: Proceedings of the ACM Web Conference 2025 (WWW '25). ACM (2025)

[11] Wu, Z., Li, X., Zhang, J., et al.: AutoAgents: An open-source framework for building multi-agent systems with large language models. arXiv preprint arXiv:2308.08155 (2023)

[12] Chinchor, N.: MUC-4 evaluation metrics. In: Proceedings of the 4th Message Understanding Conference (MUC-4), pp. 22–29. Association for Computational Linguistics (1992)

Thank You

Questions & Discussion

Huzefa Ismail Jadliwala
M.Sc. Web Engineering
Technical University of Chemnitz

EMAIL
huzefa.jadliwala@s2025.tu-chemnitz.de

DEFENSE DATE
March 18, 2026

To summarise — one question drove this thesis. Can a modular multi-agent system fill templates from speech better than a single model? The answer is yes.

Five specialised agents. Four orchestration strategies. RAG retrieval that consistently boosts performance. All at less than one cent per document.

But most importantly — Invox supports human judgment. It never replaces it.

I would like to thank Dr. Christoph Carl Kling from AIConver and Verena Traubinger from TU Chemnitz for their guidance throughout this work.

Thank you all for your time. I am happy to take your questions.

Scenario 1: Text Only (No RAG)

APPENDIX

Strategy	R1 — Consistency		R2 — Extraction Accuracy						R3	R4	R5	R6 — Usability	
	FPR ↑	SC ↑	OBS ↑	NES ↑	SF1 ↑	FDA ↑	HR ↓	MR ↓	Transparency	User Correction	Adaptation	Latency (p50)	Cost/doc
S1 — Single-Pass	0.910	0.692	0.601	0.469	0.604	0.713	0.233	0.329	✓	✓	✓	25.0s	0.72¢
S2 — Iterative	0.970	0.672	0.590	0.560	0.608	0.709	0.373	0.226	✓	✓	✓	20.0s	2.18¢
S3 — Consensus Full	0.910	0.691	0.602	0.481	0.601	0.713	0.246	0.319	✓	✓	✓	31.5s	1.40¢
S4 — Consensus Per-Field	0.870	0.660	0.549	0.628	0.589	0.693	0.551	0.112	✓	✓	✓	125.1s	4.32¢

Scenario 2:Text + RAG

APPENDIX

Strategy	R1 — Consistency		R2 — Extraction Accuracy						R3	R4	R5	R6 — Usability	
	FPR ↑	SC ↑	OBS ↑	NES ↑	SF1 ↑	FDA ↑	HR ↓	MR ↓	Transparency	User Correction	Adaptation	Latency (p50)	Cost/doc
S1 — Single-Pass	0.960	0.724	0.644	0.504	0.642	0.746	0.180	0.313	✓	✓	✓	55.9s	0.72¢
S2 — Iterative	0.950	0.683	0.619	0.555	0.639	0.718	0.301	0.267	✓	✓	✓	25.4s	2.18¢
S3 — Consensus Full	0.920	0.724	0.641	0.521	0.638	0.743	0.208	0.295	✓	✓	✓	59.4s	1.40¢
S4 — Consensus Per-Field	0.940	0.688	0.579	0.624	0.613	0.709	0.476	0.144	✓	✓	✓	121.8s	4.32¢

Scenario 3: Simulated Speech + RAG

APPENDIX

Strategy	R1 — Consistency		R2 — Extraction Accuracy						R3	R4	R5	R6 — Usability	
	FPR ↑	SC ↑	OBS ↑	NES ↑	SF1 ↑	FDA ↑	HR ↓	MR ↓	Transparency	User Correction	Adaptation	Latency (p50)	Cost/doc
S1 — Single-Pass	0.930	0.700	0.626	0.508	0.631	0.727	0.226	0.311	✓	✓	✓	49.4s	0.72¢
S2 — Iterative	0.940	0.666	0.595	0.544	0.631	0.700	0.341	0.267	✓	✓	✓	24.1s	2.18¢
S3 — Consensus Full	0.910	0.702	0.624	0.524	0.630	0.728	0.251	0.289	✓	✓	✓	53.5s	1.40¢
S4 — Consensus Per-Field	0.940	0.682	0.572	0.612	0.592	0.704	0.479	0.150	✓	✓	✓	127.2s	4.32¢

Trade-offs at a Glance

APPENDIX

STRATEGY	LLM CALLS	ERROR ISOLATION	ROBUSTNESS	LATENCY	COST
S1: Single-Pass	*1*	LOW	LOW	LOW	LOW
S2: Iterative	*n*	HIGH	MEDIUM	LOW	MEDIUM
S3: Consensus Full	*m*	LOW	HIGH	MEDIUM	HIGH
S4: Consensus Per-Field	*n × m*	HIGH	HIGH	HIGH	HIGH

Evaluation Matrix

APPENDIX

OBS Overall Baseline Score

Accuracy across ALL 9 fields including empty ones. If gold is empty AND system leaves it empty — that counts as correct too.

Example:
Gold: victim=(empty). System: victim=(empty). → OBS gets 1.0 for this field. Easy win for cautious systems.

NES Non-Empty Score

Accuracy ONLY on fields where the gold has a real value. Empty fields are completely ignored. The harder, more honest test.

Example:
Gold: weapon='bomb'. System: weapon='rifle'. → NES gets ~0.3. System tried but got it wrong. No hiding behind empty fields.

EAI Empty Advantage Index

Simply OBS minus NES. A high EAI means the system's good OBS score comes mostly from correctly leaving things empty — not real extraction.

Example:
LangExtract: OBS=0.684, NES=0.406 → EAI=0.278. High! Most of its score is from empty fields, not real extracted values.

SF1 Soft F1

Harmonic mean of Coverage (did you find all values?) and Specificity (was everything found real?). Catches both missing AND hallucinated values.

Example:
Gold: weapon=['bomb','rifle']. Pred: ['bomb','rifle','grenade'] → Coverage=1.0, Specificity=0.67 → SF1=0.80. Penalises fake 'grenade'.

Evaluation Matrix

APPENDIX